

TRABALHO 4: PROJETO FINAL COM FRAMEWORK JAVASCRIPT

Período: Semanas 7-8 (29/06 a 08/07/2026)

Valor: 15 pontos

Modalidade: Em equipe (2-3 estudantes - podem ser novas equipes)

DESCRIÇÃO

Desenvolver uma **aplicação web completa** utilizando um framework JavaScript moderno (React ou Vue.js), integrando todos os conhecimentos adquiridos ao longo da disciplina (HTML semântico, CSS responsivo, JavaScript, consumo de APIs, gerenciamento de estado).

OBJETIVOS DE APRENDIZAGEM

Ao final deste trabalho, os estudantes serão capazes de:

- Criar aplicações usando React ou Vue.js
 - Estruturar projetos com componentes reutilizáveis
 - Gerenciar estado da aplicação
 - Implementar roteamento entre páginas
 - Integrar APIs externas
 - Aplicar boas práticas de desenvolvimento moderno
 - Fazer deploy de aplicações React/Vue
-

OPÇÕES DE FRAMEWORK

OPÇÃO A: React (Recomendado)

- Create React App ou Vite
- React Hooks (useState, useEffect, etc.)
- React Router para navegação
- Context API ou props para gerenciamento de estado

OPÇÃO B: Vue.js

- Vue 3 com Composition API
- Vue Router
- Pinia ou Vuex para estado global

TIPOS DE PROJETO

OPÇÃO 1: E-commerce Simplificado

Funcionalidades Obrigatórias:

- Listagem de produtos (consumindo API ou dados mocados)
- Página de detalhes do produto
- Filtro por categoria e preço
- Busca de produtos
- Carrinho de compras funcional
- Resumo do pedido
- LocalStorage para persistir carrinho
- Responsividade completa

Páginas Mínimas:

- Home / Listagem de produtos
- Detalhes do produto
- Carrinho
- Checkout (simulado)

APIs Sugeridas:

- Fake Store API: <https://fakestoreapi.com/>
- DummyJSON: <https://dummyjson.com/>

OPÇÃO 2: Rede Social Simplificada

Funcionalidades Obrigatórias:

- Feed de posts
- Criar novo post
- Curtir e comentar posts
- Perfil de usuário
- Lista de usuários
- Busca de posts ou usuários
- LocalStorage para posts criados
- Responsividade completa

Páginas Mínimas:

- Feed/Home
- Criar Post
- Perfil de Usuário
- Listagem de Usuários

APIs Sugeridas:

- JSONPlaceholder: <https://jsonplaceholder.typicode.com/>
- ReqRes: <https://reqres.in/>

OPÇÃO 3: Gerenciador de Conteúdo (Blog/Portfólio)

Funcionalidades Obrigatórias:

- Listagem de posts/artigos
- Página individual de post
- Categorias e tags
- Busca por palavra-chave
- Comentários (simulados ou LocalStorage)
- Paginação
- Modo claro/escuro
- Responsividade completa

Páginas Mínimas:

- Home / Listagem
- Post Individual
- Categoria
- Sobre

APIs Sugeridas:

- Dev.to API: <https://developers.forem.com/api>
- NewsAPI: <https://newsapi.org/>

OPÇÃO 4: Dashboard de Dados

Funcionalidades Obrigatórias:

- Consumir API com dados numéricos
- Gráficos interativos (Chart.js ou similar)
- Filtros por período/categoria
- Cards com estatísticas

- Tabela de dados
- Exportar dados (CSV ou JSON)
- Atualização de dados em tempo real ou manual
- Responsividade completa

Páginas Mínimas:

- Dashboard Principal
- Detalhes/Relatórios
- Configurações

APIs Sugeridas:

- CoinGecko (criptomoedas): <https://www.coingecko.com/en/api>
- REST Countries: <https://restcountries.com/>
- GitHub API: <https://docs.github.com/en/rest>

OPÇÃO 5: Projeto Personalizado (Requer Aprovação)

Proposta própria desde que atenda aos requisitos mínimos de complexidade.

REQUISITOS TÉCNICOS OBRIGATÓRIOS

Componentização

- Mínimo 8 componentes React/Vue diferentes
- Componentes reutilizáveis
- Hierarquia clara de componentes
- Props sendo utilizadas adequadamente

Roteamento

- Mínimo 4 rotas diferentes
- Navegação funcional entre páginas
- 404 para rotas inexistentes
- Parâmetros de rota (ex: /produto/:id)

Gerenciamento de Estado

- State management adequado (useState, Context API, etc.)
- Estado compartilhado entre componentes quando necessário
- Estado local em componentes quando apropriado

Integração com API

- Consumo de pelo menos uma API pública
- Loading states durante requisições
- Tratamento de erros
- Exibição de dados da API em componentes

Estilização

- CSS Modules, Styled Components ou framework CSS (Bootstrap, Tailwind)
- Design responsivo
- Consistência visual
- Animações/transições (opcional mas recomendado)

Boas Práticas

- Código limpo e organizado
- Comentários em código complexo
- Nomenclatura consistente
- ESLint sem erros

ESTRUTURA DE ARQUIVOS SUGERIDA (React)

```
trabalho4/
├── public/
│   └── index.html
├── src/
│   ├── components/
│   │   ├── Header/
│   │   ├── Footer/
│   │   ├── Card/
│   │   └── ...
│   ├── pages/
│   │   ├── Home.jsx
│   │   ├── About.jsx
│   │   └── ...
│   ├── services/
│   │   └── api.js
│   ├── context/
│   │   └── AppContext.jsx
│   ├── utils/
│   │   └── helpers.js
│   ├── App.jsx
│   ├── App.css
│   └── index.jsx
├── package.json
└── README.md
```

CRITÉRIOS DE AVALIAÇÃO

Critério	Pontos	Descrição
Funcionalidades Completas	4,0	Todas as funcionalidades obrigatórias implementadas
Componentização	3,0	Uso adequado de componentes, reutilização, hierarquia
Roteamento	2,0	Navegação funcional, rotas bem definidas
Estado e Dados	2,0	Gerenciamento de estado eficiente, integração com API
Interface e Design	2,0	UI/UX de qualidade, responsividade, consistência visual
Qualidade do Código	1,0	Código limpo, organizado, boas práticas
Trabalho em Equipe	1,0	Uso do Git, contribuição balanceada, colaboração
TOTAL	15,0	

Bonificação

- **+1,0 ponto:** Deploy funcional (Vercel, Netlify, GitHub Pages)
- **+0,5 ponto:** Testes unitários implementados
- **+0,5 ponto:** Documentação excepcional (Storybook, etc.)
- **+0,5 ponto:** Funcionalidades extras criativas

Penalizações

- **-3,0 pontos:** Funcionalidade obrigatória ausente
- **-1,0 ponto:** Não responsivo
- **-1,0 ponto:** Erros no console
- **-0,5 ponto:** README incompleto

ENTREGA

Formato

- Repositório GitHub público ou privado
- Link do repositório no AVA
- README.md detalhado com:

- Descrição do projeto
- Instruções de instalação
- Tecnologias utilizadas
- Screenshots/GIFs da aplicação
- Link de deploy (se houver)

Prazo

- **Data limite:** 08/07/2026 (terça-feira) até 23:59
- **Apresentação presencial:** 08/07/2026 durante aula (4 horas)

Apresentação (15 minutos por equipe)

- Demonstração completa da aplicação
 - Explicação da arquitetura de componentes
 - Mostrar código de componente interessante
 - Explicar decisões técnicas e desafios
 - Deploy ao vivo (se disponível)
-

RECURSOS DE APOIO

React

- React Docs (oficial): <https://react.dev/>
- Create React App: <https://create-react-app.dev/>
- Vite: <https://vitejs.dev/>
- React Router: <https://reactrouter.com/>

Vue.js

- Vue 3 Docs: <https://vuejs.org/>
- Vue Router: <https://router.vuejs.org/>
- Pinia: <https://pinia.vuejs.org/>

Ferramentas

- Component libraries: Material-UI, Ant Design, Chakra UI
- Chart libraries: Chart.js, Recharts
- Icon libraries: React Icons, Heroicons
- Deploy: Vercel, Netlify, GitHub Pages

Tutoriais

- freeCodeCamp - React: <https://www.freecodecamp.org/learn/front-end-development-libraries/>
 - Scrimba - React: <https://scrimba.com/learn/learnreact>
 - Vue Mastery: <https://www.vuemastery.com/>
-

DICAS FINAIS

1. **Comece Simples:** Implemente funcionalidades básicas primeiro, depois adicione complexidade
 2. **Componentize:** Pense em componentes reutilizáveis desde o início
 3. **State Management:** Não complique — comece com state local e eleve apenas quando necessário
 4. **Console Errors:** Mantenha o console limpo durante desenvolvimento
 5. **Git Flow:** Use branches para features, merges frequentes
 6. **Deploy Cedo:** Faça deploy cedo para identificar problemas
 7. **Teste Constantemente:** Teste em diferentes navegadores e dispositivos
 8. **Documentem:** Mantenham README atualizado durante desenvolvimento
 9. **Perguntem:** Não esperem até o último dia para tirar dúvidas!
-

Boa sorte e bom desenvolvimento!